

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

3.1 Архитектура программного модуля

Программный модуль телеграм-бота с поддержкой языка жестов представлен следующими основными функциональными блоками:

- блок нейронной сети для распознавания языка жестов;
- блок обучения нейронной сети;
- блок подготовки данных перед обработкой с помощью нейронной сети;
- блок интеграции с телеграм-ботом.

Описание архитектуры каждого из блоков представлено ниже.

3.2 Блок нейронной сети

Нейронная сеть, предназначенная для распознавания языка жестов, реализует концепцию «последовательность-в-последовательность» и состоит из двух ключевых компонентов: энкодера и декодера. Каждый компонент включает в себя специализированные слои и подмодули, обеспечивающие извлечение пространственно-временных признаков из видеопоследовательностей, и преобразование их в текстовую расшифровку.

3.1.1 Энкодер

Энкодер предназначен для обработки входной последовательности графов, где каждый кадр видео представлен в виде набора ключевых точек руки с координатами x , y и z . Он состоит из двух основных частей:

Класс `GCNLayer`. `GCNLayer`, или графовый сверточный слой выполняет агрегацию признаков ключевых точек с учётом их пространственных взаимосвязей, используя матрицу смежности, описывающую топологию руки. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, in_features, out_features)` – инициализирует линейное преобразование, задавая размеры входных и выходных признаков. Принимаемые параметры:

- `in_features`: количество входных признаков, например, 3 для координат x , y , z ;
- `out_features`: количество выходных признаков, или же размер скрытого слоя.

2. Метод `forward(self, x, adj)` – перемножает тензор с данными о ключевых точках и матрицу смежности, и пропускает результат через линейное преобразование с активацией `ReLU` для усиления нелинейных зависимостей. Принимаемые параметры:

- `x`: тензор признаков узлов;
- `adj`: матрица смежности, описывающая связи между узлами.

Класс Encoder. Этот класс отвечает за обработку временной динамики последовательности кадров. Данный модуль обрабатывает каждый кадр видеопоследовательности: сначала для каждого кадра применяется GCNLayer, затем выполняется глобальное усреднение по всем ключевым точкам, чтобы получить компактное представление кадра, после чего последовательность этих векторов обрабатывается LSTM для учета временной зависимости жестов. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, node_feature_dim, hidden_dim, num_nodes)` задаёт ключевые параметры и инициализирует GCNLayer и LSTM. Принимаемые параметры:

- `node_feature_dim`: размерность признаков узлов;
- `hidden_dim`: размер скрытого состояния;
- `num_nodes`: количество ключевых точек.

2. Метод `forward(self, x_seq, adj)` – проходит по каждому кадру входной последовательности, применяя GCNLayer для извлечения локальных признаков, затем выполняет усреднение по всем точкам и передаёт полученную последовательность в LSTM для формирования скрытых представлений, которые затем используются декодером. Принимаемые параметры:

- `x_seq`: тензор последовательности кадров;
- `adj`: матрица смежности.

3.1.2 Декодер

Декодер отвечает за преобразование скрытых представлений, полученных от энкодера, в последовательность токенов, которая интерпретируется как текстовая расшифровка жестов. Он включает следующие ключевые компоненты:

Класс Decoder. Decoder отвечает за генерацию текстовой последовательности. Этот модуль сначала преобразует входную последовательность токенов в числовые векторы, называемые эмбедингами, с помощью одноименного слоя, затем обрабатывает их через LSTM, а в конце применяет линейное преобразование для получения распределения вероятностей по словарю. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, hidden_dim, vocab_size, embedding_dim)` – настраивает слой эмбединга (для представления токенов в виде векторов), LSTM для последовательной обработки и линейный слой, выводящий логиты по каждому элементу словаря. Принимаемые параметры:

- `node_feature_dim`: размер скрытого состояния;
- `hidden_dim`: размер словаря токенов;
- `num_nodes`: размер эмбедингов токенов.

2. Метод `forward(self, target_seq, hidden, cell)` – принимает входной текст в виде последовательности токенов и скрытые состояния, преобразует токены в эмбединги, затем обрабатывает их через LSTM, после

чего результат пропускается через линейный слой, который возвращает вероятность выбора каждого токена. Принимаемые параметры:

- `target_seq`: последовательность целевых токенов;
- `hidden`: скрытое состояние от энкодера;
- `cell`: ячейка состояния от энкодера.

Класс `Seq2Seq`. Этот класс объединяет работу энкодера и декодера, позволяя обучать модель с использованием техники `teacher forcing`. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, encoder, decoder, device)` – принимает готовые экземпляры энкодера и декодера, а также указывает устройство вычислений – CPU или GPU. Принимаемые параметры:

- `encoder`: экземпляр класса `Encoder`;
- `decoder`: экземпляр класса `Decoder`;
- `device`: устройство вычислений.

2. Метод `forward(self, src, adj, trg)` – последовательно пропускает входные данные через энкодер, затем использует скрытые состояния для запуска декодера, который генерирует итоговую последовательность токенов. Принимаемые параметры:

- `src`: входная последовательность кадров;
- `adj`: матрица смежности;
- `trg`: целевая последовательность токенов.

Класс `SignLanguageModel`. Этот класс служит интерфейсом для работы с моделью, обеспечивая удобную загрузку обученных весов, выполнение жадного декодирования и получение окончательного текстового вывода. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, model_path, vocab, node_feature_dim=3, hidden_dim=64, embedding_dim=32, num_nodes=21, device=None)` – инициализирует энкодер, декодер и `Seq2Seq` модели, загружает веса из файла и устанавливает словарь для токенизации. Принимаемые параметры:

- `model_path`: путь к файлу модели;
- `vocab`: словарь токенов;
- `node_feature_dim`: размерность признаков узлов;
- `hidden_dim`: размер скрытого состояния;
- `embedding_dim`: размер эмбедингов;
- `num_nodes`: количество ключевых точек;
- `device`: устройство.

2. Метод `load_model(self)` – отвечает за загрузку сохранённого состояния модели из указанного файла, проверяя наличие файла и устанавливая веса. Принимаемых параметров, кроме экземпляра класса, нет.

3. Метод `greedy_decode(self, src, adj, max_length=20)` – реализует алгоритм жадного декодирования, при котором генерируется последовательность токенов до появления специального токена конца

последовательности или до достижения максимальной длины. Принимаемые параметры:

- `src`: входная последовательность кадров;
- `adj`: матрица смежности;
- `max_length`: максимальная длина последовательности.

4. Метод `predict(self, video_array, adj, max_length=20)` – принимает массив ключевых точек, добавляет размерность батча, выполняет инференс через модель и возвращает текстовую расшифровку жестов, формируя её из сгенерированных токенов по обратному отображению словаря. Принимаемые параметры:

- `video_array`: массив ключевых точек;
- `adj`: матрица смежности;
- `max_length`: максимальная длина.

3.2 Блок подготовки данных

Блок подготовки данных играет ключевую роль в получении и предварительной обработке информации из видео. Классов в данном блоке нет, однако есть функции, выполняющие определенную роль. Такие ключевые функции описаны ниже:

1. Функция `extract_keypoints_from_video` использует MediaPipe для обработки видео кадр за кадром. Для каждого кадра определяется, обнаружена ли рука, и если да, то извлекаются 21 ключевая точка, представленные координатами *x*, *y*, *z*. В случае отсутствия обнаружения используется нулевой вектор. Принимаемые параметры:

- `video_path`: путь к видео;
- `target_seq_len`: целевая длина;
- `num_keypoints`: количество ключевых точек.

2. Функция `process_mediapipe_data`. Для корректной работы модели полученная последовательность кадров либо дополняется нулевыми значениями, если её длина меньше заданного значения, либо усечается до требуемой длины, что гарантирует единообразие входных данных. Принимаемые параметры:

- `json_file`: путь к JSON-файлу;
- `target_seq_len`: целевая длина последовательности;
- `num_keypoints`: количество ключевых точек.

3. Функция `create_hand_adjacency_matrix` формирует матрицу смежности, описывающую пространственные связи между ключевыми точками руки. Эта матрица затем используется в графовом сверточном слое энкодера для эффективного агрегирования пространственных признаков. Принимаемые параметры:

- `num_keypoints`: количество ключевых точек.

3.3 Блок обучения

Класс `MediaPipeSignLanguageDataset`. Класс играет ключевую роль в процессе подготовки данных для обучения модели распознавания жестового языка. Его основная задача – загрузить данные из JSON-файлов, содержащих координаты ключевых точек для каждого кадра видео, и соответствующие текстовые метки. Эти данные затем преобразуются в формат, который может быть использован для обучения нейронной сети, работающей с последовательностями и графами. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, json_paths, labels, target_seq_len, num_keypoints, vocab)`. Этот метод инициализирует экземпляр набора данных, подготавливая все необходимые структуры данных для дальнейшей работы. Он сохраняет переданные параметры, такие как пути к файлам, метки и настройки обработки, а также создаёт матрицу смежности — структуру, которая описывает связи между ключевыми точками руки, например, между пальцами или суставами. Принимаемые параметры:

- `json_paths`: список путей к JSON-файлам;
- `labels`: список меток;
- `target_seq_len`: длина последовательности кадров;
- `num_keypoints`: количество ключевых точек;
- `vocab`: словарь токенов.

3.4 Блок интеграции с Telegram-ботом

Для обеспечения интерактивного взаимодействия с пользователем разработан модуль, интегрирующий модель распознавания жестов с Telegram-ботом. Основные функции этого блока таковы:

Приём видео от пользователя. Бот, реализованный с использованием библиотеки `python-telegram-bot`, обрабатывает входящие видео-сообщения. При получении сообщения бот извлекает файл видео, сохраняет его локально и уведомляет пользователя о начале обработки.

Извлечение признаков и инференс. После получения видео вызывается функция из модуля `dataprocessor`, которая извлекает ключевые точки из видео. Затем, с помощью метода `predict` класса `SignLanguageModel`, производится инференс модели, который генерирует текстовую расшифровку жестов.

Отправка результата пользователю. Полученный текст отправляется обратно пользователю в виде сообщения, обеспечивая оперативную обратную связь.

Модуль Telegram-бота организован в файле `telegram_bot`, где подробно прописаны обработчики команд и видео-сообщений, а также настройки подключения к Telegram API.

3.4 Зависимости и организация проекта

Проект структурирован модульно для обеспечения простоты поддержки и расширения функциональности:

Модуль `model` содержит классы, реализующие архитектуру нейронной сети – `GCNLayer`, `Encoder`, `Decoder`, `Seq2Seq` – а также обёртку `SignLanguageModel`, предоставляющую удобный интерфейс для инференса

Модуль `dataprocessor` включает функции для обработки входных данных: извлечение ключевых точек из видео и построение матрицы смежности

Модуль `telegram_bot` отвечает за интеграцию модели с Telegram-ботом, организуя получение видео, запуск инференса и отправку результатов пользователю

Эта организация позволяет независимо тестировать и дорабатывать каждый компонент, легко интегрировать новые методы обработки или изменять архитектуру модели, а также обеспечивает масштабируемость и повторное использование кода в других приложениях.